

Universidad del Valle de Guatemala

Facultad de Ingeniería

Departamento de Ciencias de la Computación

Construcción de Compiladores

[M3] Fase de Compilación: Generación de Código MIPS

Backend MIPS32 y ejecución en MARS para Compiscript

ANTLR 4.13.2 • C++17 • TAC • MIPS32 • MARS 4.5 • IDE web

`https://github.com/G1LB3TO/Proyecto1_Compis_Generador_Analizador_Lexico/tree/Compiladores2-Proyecto3-MIPS`

Catedrático

Ing. Pablo Koch

Integrantes

Luis Gonzalez — 23353

Joel Jaquez — 23369

Guatemala, 8 de noviembre de 2026

Versión del código documentada: `a82cd63`

Índice

1. Introducción **3**

2. Objetivos **3**

2.1. Objetivo general 3

2.2. Objetivos específicos 3

3. Arquitectura general **4**

3.1. El pipeline 4

3.2. Una tercera pasada sobre cuádruplos 4

3.3. Núcleo independiente de la interfaz 5

3.4. Cuándo se genera código 5

4. Estructura del repositorio y trazabilidad por commits **5**

4.1. Historial de la rama 5

4.2. Lectura del historial 7

5. Descripción de los archivos por capa **7**

5.1. Núcleo del compilador 8

5.2. Simulador, pruebas, IDE y documentación 8

6. Representación física de valores **8**

7. Convención de llamadas y registros de activación **10**

7.1. Organización del programa en rutinas 10

7.2. Registro de activación físico 10

7.3. Responsabilidades del llamante 10

7.4. Responsabilidades del llamado 10

7.5. Acceso a datos no locales 11

7.6. Recursividad 12

8. Selección de instrucciones **12**

8.1. Ejemplo: una suma 13

8.2. Entorno de ejecución 13

9. Asignación de registros **14**

9.1. Registros disponibles 14

9.2. Política de `getReg` 14

9.3. Relación con los temporales del TAC 15

9.4. Métricas 15

10. Errores de ejecución y `try/catch` 15

11. Integración con MARS y código máquina 15

11.1. Preparación del simulador 15

11.2. Ejecución 16

11.3. Código máquina 16

12. Interfaz gráfica (IDE) 16

13. Batería de pruebas 17

13.1. Pruebas de ejecución en MARS 17

13.2. Prueba de fuego: algoritmos del propio compilador 18

13.3. Ejecución de las baterías 19

14. Compilación y ejecución 19

14.1. Requisitos 19

14.2. Construcción 19

14.3. Ejecución del IDE 19

14.4. Uso directo del CLI 20

15. Decisiones, supuestos y limitaciones 20

16. Correspondencia con la rúbrica 21

17. Conclusiones 21

18. Referencias 22

1. Introducción

Este informe documenta la fase final del compilador de **Compiscript**: la **generación de código MIPS**. La fase parte del código de tres direcciones (TAC) y de la tabla de símbolos extendida que produjo la fase anterior, y los traduce a **ensamblador MIPS32** que el simulador **MARS** ensambla a código máquina y ejecuta. Con ello el compilador recorre, por primera vez, el camino completo desde un archivo `.cps` hasta un programa que corre y produce salida observable.

Como en las fases anteriores, **todo el compilador está escrito en C++17**. El backend es una tercera pasada, el `MipsGenerator`, que se enlaza en la misma biblioteca nativa que el analizador semántico y el generador de TAC. Python aparece únicamente como adaptador: el servidor Flask del IDE y un módulo que invoca a MARS y recoge su salida. Ninguna regla de traducción a MIPS está implementada fuera de C++.

La fase aporta cuatro piezas, que corresponden a los componentes evaluados:

- **Un generador MIPS32** que traduce cada familia de cuádruplos —datos, expresiones, saltos, funciones, listas, objetos, campos, métodos y excepciones— a instrucciones, junto con un entorno de ejecución propio para aritmética mixta, cadenas, impresión y errores.
- **Una convención de llamadas** con registros de activación físicos de 64 bytes de cabecera, enlaces de control y de acceso, paso de parámetros por pila y retorno en `$v0`/`$v1`.
- **Un algoritmo de asignación de registros** basado en `getReg`, con descriptores de registros, política LRU y *spills* a la pila, con métricas observables.
- **La integración con MARS**: ensamblado, ejecución con límite de pasos, volcado del código máquina y una consola de ejecución en el IDE.

La validación crece a **193 casos automatizados**: 115 escenarios C++ de semántica, TAC y MIPS, 66 programas que se ejecutan de verdad en MARS y cuya salida se compara con un resultado esperado, y 12 pruebas de integración del IDE.

2. Objetivos

2.1. Objetivo general

Implementar un generador de código objeto para Compiscript que, a partir del código de tres direcciones y la tabla de símbolos de las fases anteriores, emita ensamblador MIPS32 correcto y ejecutable en un simulador, manejando llamadas, retornos, registros de activación y asignación de registros.

2.2. Objetivos específicos

- Definir una representación física de los valores del lenguaje sobre palabras de 32 bits, que conserve el tipo efectivo de cada valor.
- Diseñar e implementar una convención de llamadas con prólogo, epílogo, enlaces de control y de acceso, paso de parámetros y retorno.
- Traducir cada instrucción del TAC a una secuencia MIPS equivalente, incluidas funciones anidadas, recursión, clases, herencia, listas y `try/catch`.

- Implementar un algoritmo `getReg` con descriptores de registros y *spills*, y exponer sus métricas.
- Ejecutar el ensamblador en MARS, distinguir errores de ensamblado y de ejecución, y empaquetar el código máquina resultante.
- Ampliar el IDE y las baterías de pruebas para compilar, ejecutar y verificar los programas generados.

3. Arquitectura general

3.1. El pipeline

El flujo de información va del texto fuente al programa ejecutado en MARS. Las etapas nuevas de esta fase son el `MipsGenerator` y el adaptador del simulador:

```
Código .cps
    |
    v
CompiscriptLexer / CompiscriptParser  (ANTLR, C++)
    |
    v
SemanticAnalyzer  (Visitor C++, primera pasada)
    |-- tipos, ámbitos, tabla de símbolos
    v
TacGenerator      (Visitor C++, segunda pasada)
    |-- cuádruplos tipados + registros de activación
    v
(solo si léxico, sintaxis, semántica y TAC son correctos)
    v
MipsGenerator     (C++, tercera pasada)        <<< NUEVO
    |-- rutinas, frames físicos, getReg, runtime
    v
JSON: diagnósticos + símbolos + TAC + MIPS + frames
    |
    v
Flask (adaptador) -> IDE web (Código MIPS, Consola MIPS)
    |
    v
mips_runtime.py -> MARS 4.5 (ensamblado y ejecución)   <<< NUEVO
```

3.2. Una tercera pasada sobre cuádruplos

A diferencia del analizador semántico y del generador de TAC, el `MipsGenerator` **no es un Visitor del árbol**. Recibe el `SemanticResult` y el `TacResult` y trabaja sobre los cuádruplos estructurados: usa `op`, `arg1`, `arg2` y `result` de cada `TacInstruction`. La representación textual del TAC es solo una vista para lectura humana; nunca se vuelve a tokenizar.

Para seleccionar instrucciones y aplicar coerciones, el generador necesita el tipo de cada operando. Por eso esta fase extiende `TacInstruction` con tres campos nuevos —`result_type`, `arg1_type` y `arg2_type`— que el generador de TAC llena al emitir cada cuádruplo.

3.3. Núcleo independiente de la interfaz

El núcleo sigue exponiendo una sola función pública, `analyzeSource`, que devuelve un `AnalysisResult`. Ahora incluye el bloque `mips` (`MipsResult`) y el indicador `mips_ok`. El CLI, las baterías de pruebas y el IDE usan el mismo servicio, de modo que lo que se prueba es exactamente lo que se ejecuta.

3.4. Cuándo se genera código

El backend **solo se ejecuta cuando todas las fases anteriores terminaron sin errores**. Si alguna falla, el resultado MIPS queda marcado como `skipped` con una explicación en `reason`:

Entrada	Sintaxis	Semántica	TAC	MIPS
Error léxico o sintáctico	con errores	omitida	omitido	omitido
Error semántico	correcta	con errores	omitido	omitido
Error de generación MIPS	correcta	correcta	generado	con errores
Programa válido	correcta	correcta	generado	generado

Tabla 1: Estado de cada fase según el punto en que falla el programa.

Un error de generación MIPS —por ejemplo, un literal entero que no cabe en 32 bits— tiene código, categoría *generación MIPS*, mensaje, línea y columna, igual que los diagnósticos de las fases anteriores. Además, los resultados de **compilación** y **ejecución** son distintos: un programa puede compilar correctamente y fallar al ejecutarse (división por cero, índice fuera de rango), y el IDE los presenta por separado.

4. Estructura del repositorio y trazabilidad por commits

El desarrollo de esta fase se realizó en la rama `Compiladores2-Proyecto3-MIPS`, a partir del último commit de la fase de código intermedio (`4b379ad`). Cada commit corresponde a una unidad funcional completa y a un solo autor, de modo que la contribución individual queda registrada en el historial. Los ocho commits se publicaron el 8 de noviembre de 2026 y suman 26 archivos modificados, 3 056 líneas agregadas y 108 eliminadas. Las identidades de Git corresponden a los dos integrantes:

Identidad en Git	Correo	Integrante
`G1LB3T0`	luis.yo09@hotmail.com	Luis Gonzalez (23353)
`jaq23369`	jaq23369@uvg.edu.gt	Joel Jaquez (23369)

Tabla 2: Identidades de Git utilizadas en la rama.

4.1. Historial de la rama

La tabla siguiente recorre los ocho commits de la fase en orden cronológico y explica qué archivos introduce cada uno y por qué. Es, en la práctica, el mapa de lectura del repositorio.

Commit	Autor	Aporte y archivos
`afe1463`	Luis Gonzalez	**[ADD] Implementar modelo y generador MIPS de Compiscript.** El corazón de la fase. Crea `mips_generator.h` y `mips_generator.cpp` (1 012 líneas): extracción de rutinas, *layout* de frames físicos, `getReg`, selección de instrucciones y el entorno de ejecución en ensamblador. En `model.h` agrega `MipsFrame`, `MipsResult`, los tipos de operandos de `TacInstruction` y `AnalysisResult.mips`. 3 archivos, +1 062/−0.
`402a3c2`	Luis Gonzalez	**[UPDATE] Preparar semántica y TAC para generación MIPS.** Ajusta las fases anteriores a lo que el backend necesita: el TAC registra los tipos de cada operando y conserva el `this` léxico de funciones anidadas en métodos; la semántica impide que `break`/`continue` atraviesen una función anidada y exige retorno en `try` y en `catch`. Agrega los escenarios correspondientes a `compiscript_tests.cpp`. 4 archivos, +223/−34.
`50a4dbf`	Luis Gonzalez	**[ADD] Integrar backend MIPS en servicio y CLI.** En `service.cpp` invoca al `MipsGenerator` solo si el TAC es válido. En `json.cpp` serializa el bloque `mips` (texto, diagnósticos, métricas y frames). En `main.cpp` agrega `-emit-mips` y `-o` para guardar el ensamblador. 4 archivos, +105/−15.
`db2e0fa`	Joel Jaquez	**[ADD] Ejecutar y ensamblar MIPS con MARS.** Crea `mips_runtime.py` (invocación de MARS, límite de pasos, volcados y paquete ZIP), `scripts/run_mips.py` (CLI de ejecución) y `scripts/setup_mars.py` (descarga verificada por SHA-256). Integra la preparación de MARS en los scripts de construcción. 5 archivos, +233/−0.
`03408a6`	Joel Jaquez	**[TEST] Validar ejecución MIPS y algoritmos del compilador.** Crea `mips_execution_tests.py` (372 líneas, 66 casos que se ejecutan en MARS) y el programa `algoritmo_compilador.cps`; registra la batería en CTest cuando hay Python, Java y MARS. 3 archivos, +464/−0.
`279ddee`	Joel Jaquez	**[ADD] Integrar MIPS y consola de ejecución en el IDE.** Agrega a `app.py` las rutas `/api/simulator`, `/api/run` y `/api/build`; al IDE, las pestañas *Código MIPS* y *Consola MIPS*, el botón *Ejecutar en MIPS* y el estado del simulador. Crea `ide_integration_tests.py` con 12 pruebas. 5 archivos, +374/−24.

Commit	Autor	Aporte y archivos
`bd0d6b6`	Joel Jaquez	**[DOC] Documentar arquitectura y generación MIPS.** Crea `docs/generacion-mips.md` (399 líneas) con la representación de valores, la convención de llamadas, la política de `getReg`, el uso de MARS y las pruebas; actualiza `docs/arquitectura-compiscript.md`. 2 archivos, +465/−5.
`a82cd63`	Luis Gonzalez	**[DOC] Actualizar guía de uso y pruebas del proyecto 3.** Reescribe el `README.md` con la arquitectura completa, los requisitos (Java y MARS), la construcción, el uso del CLI, del IDE y de las tres baterías. 1 archivo, +130/−30.

Tabla 3: Trazabilidad de la rama `Compiladores2-Proyecto3-MIPS`.

4.2. Lectura del historial

El orden de los commits refleja el orden natural de construcción de la fase. Primero se implementa el generador y su modelo de datos (`afe1463`) y se ajustan las fases anteriores a lo que necesita (`402a3c2`); luego se conecta al servicio y al CLI (`50a4dbf`) y se ejecuta en el simulador real (`db2e0fa`). Después llegan la evidencia de que el código generado funciona (`03408a6`), la interfaz (`279ddee`) y la documentación (`bd0d6b6`, `a82cd63`). La autoría se reparte entre los dos integrantes: cuatro commits cada uno.

5. Descripción de los archivos por capa

La fase no modifica la gramática ni el código generado por ANTLR. Las tablas siguientes listan los archivos nuevos o modificados, con su tamaño actual en líneas.

5.1. Núcleo del compilador

Archivo	Estado	Líneas	Función
`src/mips_generator.cpp`	nuevo	1012	Tercera pasada: rutinas, frames, `getReg`, selección de instrucciones y entorno de ejecución.
`include/.../mips_generator.h`	nuevo	19	Interfaz pública `MipsGenerator::generate()`.
`include/.../model.h`	modificado	217	`MipsFrame`, `MipsResult` y tipos de operandos del TAC.
`src/tac_generator.cpp`	modificado	1314	Emite los tipos de operandos y el `this` léxico.
`src/semantic_analyzer.cpp`	modificado	1288	Límites de `break`/`continue` y retorno en `try/catch`.
`src/service.cpp`	modificado	101	Encadena semántica, TAC y MIPS.
`src/json.cpp`	modificado	330	Serializa el bloque `mips`.
`src/main.cpp`	modificado	96	Opciones `-emit-mips` y `-o`.

Tabla 4: Archivos del núcleo C++ tocados en esta fase.

5.2. Simulador, pruebas, IDE y documentación

Archivo	Líneas	Función
`mips_runtime.py`	125	Adaptador de MARS: ejecución, volcados y paquete de código máquina.
`scripts/run_mips.py`	66	CLI para compilar y ejecutar en MARS.
`scripts/setup_mars.py`	37	Descarga y verificación de `Mars.jar`.
`tests/mips_execution_tests.py`	372	66 programas ejecutados en MARS con salida esperada.
`tests/ide_integration_tests.py`	102	12 pruebas de las rutas Flask.
`tests/compiscript_tests.cpp`	870	115 escenarios de semántica, TAC y MIPS.
`examples/.../algoritmo_compilador.cps`	79	Algoritmos del propio compilador en Compiscript.
`app.py`	219	Rutas de ejecución, ensamblado y estado del simulador.
`frontend/static/js/app.js`	1247	Pestañas *Código MIPS* y *Consola MIPS*.
`docs/generacion-mips.md`	399	Diseño del backend y uso de MARS.
`README.md`	397	Compilación, ejecución y pruebas.

Tabla 5: Archivos del simulador, pruebas, interfaz y documentación.

6. Representación física de valores

El destino es **MIPS32**: las direcciones y las palabras son de 32 bits. Compiscript tiene tipos estáticos, pero algunas situaciones requieren conocer el tipo en tiempo de ejecución: un temporal `%tN` reciclado puede contener valores de tipos distintos en instrucciones sucesivas, la variable de un `catch` es de tipo desconocido y `print` debe formatear cada valor según su tipo. Por eso

cada valor ocupa un slot de 8 bytes: una palabra de carga útil seguida de una palabra de etiqueta de tipo.

Etiqueta	Tipo	Carga útil
0	`null`	Cero.
1	`integer`	Entero con signo de 32 bits.
2	`boolean`	Cero o uno.
3	`string`	Dirección de una cadena UTF-8 terminada en cero.
4	`float`	Dirección de un doble de 64 bits alineado a 8 bytes.
6	Lista	Dirección de su cabecera.
7	Objeto	Dirección de su cabecera.
8	Función	Dirección de su código.

Tabla 6: Etiquetas de tipo de un slot de valor.

Los dobles se almacenan **por referencia** para que el slot mantenga el mismo tamaño que los demás valores. Los literales flotantes residen en `.data`; los resultados calculados se alojan en memoria dinámica. La ampliación `integer` → `float` se realiza al almacenar en un destino flotante, al pasar parámetros y al devolver valores de ese tipo.

Rangos y aritmética.

- Los literales enteros deben estar entre $-2\,147\,483\,648$ y $2\,147\,483\,647$; si no, el backend emite `MIPS_INTEGER_RANGE`.
- La suma, resta, multiplicación y negación enteras conservan los 32 bits bajos del resultado.
- Los dobles deben ser finitos: un literal fuera de rango se diagnostica al compilar (`MIPS_FLOAT_RANGE`) y un resultado no finito se reporta al ejecutar, en lugar de propagarse como infinito o NaN.
- La división produce un resultado flotante; la división o el resto por cero producen un error de ejecución capturable con `try/catch`.

Cadenas, listas y objetos.

- Las cadenas son bytes UTF-8. Las secuencias `\n`, `\r`, `\t`, `\0`, `\"` y `\\` se decodifican al construir el literal. La salida usa el servicio 15 de MARS (escritura en descriptor) para no partir los caracteres multibyte.
- Una lista tiene una cabecera de 8 bytes con la longitud en el offset 0; el elemento i está en `base + 8 + 8·i`. Una lista anidada guarda referencias a otras listas.
- Un objeto tiene una cabecera de 8 bytes con una referencia al nombre de la clase; un campo está en `base + 8 + offset_TAC`, y los campos de la clase base preceden a los de la derivada. Los inicializadores de campos se ejecutan para cada instancia antes del constructor.

La memoria dinámica se obtiene con el servicio 9 (`sbrk`) del simulador, se inicializa en cero y se conserva durante toda la ejecución; esta fase no incorpora un recolector de basura.

7. Convención de llamadas y registros de activación

7.1. Organización del programa en rutinas

Antes de emitir `.text`, el generador recorre los cuádruplos y separa **rutinas**: el programa principal (`main`), cada función o método delimitado por `begin_func`/`end_func` y un inicializador `__init_<Clase>` por cada clase. Una pila de rutinas abiertas permite que una función declarada dentro de otra se extraiga como rutina propia, sin que su cuerpo se ejecute en el punto de la declaración. Así las funciones pueden declararse antes o después de sus llamadas.

7.2. Registro de activación físico

Cada rutina reserva un frame alineado a 8 bytes. `$fp` apunta a su inicio y `$sp` delimita el espacio de la activación y de las llamadas que realiza. La cabecera ocupa 64 bytes:

Offset desde `$fp`	Contenido
0	`$fp` anterior: **enlace de control**.
4	Frame del padre léxico: **enlace de acceso**.
8	Dirección de retorno `$ra`.
12–40	Valores preservados de `$s0` a `$s7`.
44	Manejador de errores activo al entrar en la rutina.
48	Carga útil del receptor `this`, cuando existe.
52	Etiqueta del receptor.
56–63	Reservado.
64 en adelante	Slots de 8 bytes para parámetros copiados, locales y temporales.

Tabla 7: Cabecera del registro de activación físico.

El tamaño del frame es $64 + 8 \cdot n$, donde n es el número de nombres distintos (variables locales, parámetros y temporales) que la rutina usa. Cada nombre recibe un slot estable en el orden en que aparece, de modo que un valor sacado de un registro siempre tiene dónde guardarse. El generador publica estos datos en `MipsFrame`: `frame_size`, `parameter_bytes`, `local_bytes`, `temporary_bytes` y `saved_register_bytes` (los 64 bytes de cabecera).

7.3. Responsabilidades del llamante

1. Vacía los registros modificados a sus slots (`flush`).
2. Reserva $8 \cdot n$ bytes en la pila y escribe allí los argumentos, de izquierda a derecha; en un método, el receptor precede a los argumentos explícitos. Si el parámetro formal es `float`, aplica la promoción antes de escribirlo.
3. Coloca en `$a0` el frame del **padre léxico** de la rutina destino, recorriendo la cadena de enlaces de acceso desde su propio frame.
4. Salta con `jal` y, al volver, retira el área de argumentos y copia `$v0`/`$v1` al destino del cuádruplo.

7.4. Responsabilidades del llamado

El prólogo y el epílogo que emite `Generator::routine` son, para una rutina cuyo frame mide `F` bytes:

```
fn_etiqueta:
  addiu $sp, $sp, -F        # reserva el frame
  sw $fp, 0($sp)            # enlace de control
  sw $a0, 4($sp)            # enlace de acceso recibido del llamante
  sw $ra, 8($sp)            # dirección de retorno
  sw $s0, 12($sp)           # ... $s0-$s7 en 12..40
  lw $t8, __handler
  sw $t8, 44($sp)           # manejador de errores de entrada
  move $fp, $sp
  sw $zero, 64($fp)         # cada slot inicia en null
  ...
  # cuerpo: TAC traducido
  ...
fn_etiqueta__return:
  lw $t8, 44($fp)
  sw $t8, __handler         # restaura el manejador de entrada
  move $sp, $fp
  lw $ra, 8($sp)
  lw $s0, 12($sp)           # ... restaura $s0-$s7
  lw $fp, 0($sp)
  addiu $sp, $sp, F
  jr $ra
  nop
```

Listing 1: Prólogo y epílogo generados para cada rutina.

Los parámetros se leen desde el área del llamante, que está justo encima del frame: el parámetro i está en `$fp + F + 8·i` (el índice incluye el receptor implícito en los métodos). Los métodos e inicializadores copian el receptor a `$s0` y al offset 48. El retorno utiliza `$v0` para la carga útil y `$v1` para la etiqueta; todo `return` salta a la etiqueta `__return` común, y una rutina que termina sin `return` devuelve `null`.

7.5. Acceso a datos no locales

Cuando una instrucción usa una variable de una función exterior, el generador calcula su dirección con la rutina `ancestor`: parte de `$fp` y sigue el enlace de acceso (offset 4) tantas veces como niveles léxicos haya entre la rutina actual y la propietaria del símbolo; luego suma el offset del slot en el frame del propietario.

```
move $t8, $fp
lw $t8, 4($t8)          # frame del padre léxico
lw $t8, 4($t8)          # frame del abuelo léxico
addiu $t8, $t8, 72      # slot de la variable capturada
lw $s1, 0($t8)          # carga útil
lw $s2, 4($t8)          # etiqueta
```

Listing 2: Acceso a una variable dos niveles léxicos afuera.

El **padre de definición puede ser distinto del llamante**: una llamada entre funciones hermanas o una llamada recursiva recibe en `$a0` el entorno exterior que le corresponde léxicamente, no el frame de quien la invoca. Una función declarada dentro de un método accede a su `this` léxico copiándolo del frame del método, sin convertirse en un método con un argumento receptor adicional.

7.6. Recursividad

Cada llamada crea una activación independiente: los temporales y variables de una llamada recursiva viven en su propio frame y no comparten almacenamiento con la activación que la invocó. Los registros `$s0`–`$s7` y la dirección de retorno se preservan en la cabecera, de modo que los valores que deben sobrevivir a una llamada (por ejemplo, `n` en `n * factorial(n - 1)`) se recuperan intactos al volver.

8. Selección de instrucciones

Cada cuádruplo se traduce en `Generator::instruction`. Antes de cada traducción se emite un comentario `# TAC n: ...` con el texto del cuádruplo original, de modo que el ensamblador puede leerse en paralelo con el TAC.

Operación TAC	Traducción MIPS
`declare`	Carga el valor por defecto del tipo (`0`, `false` o `null`) y lo escribe en el destino.
`assign`	Copia carga útil y etiqueta del origen al destino, con promoción a `float` si el destino lo requiere.
`add`, `sub`, `mul`, `div`, `mod`	Operandos en `$a0`–`$a3`; `jal __add` (etc.). La rutina auxiliar decide aritmética entera, flotante o concatenación según las etiquetas.
`eq`, `ne`, `lt`, `le`, `gt`, `ge`	`jal __eq` (etc.), sobre `__compare`: compara números mixtos y cadenas por contenido y orden lexicográfico.
`neg`, `not`	`jal __neg` / `__not`.
`label`	Vacía registros y emite la etiqueta.
`goto`	Vacía registros y emite `j L`; si el salto sale de un `try`, retira antes los manejadores.
`if_true`, `if_false`	Carga la condición y emite `bne`/`beq $a0, $zero, L`.
`print`	`jal __print` y salto de línea con el servicio 4.
`param`, `call`, `call_method`	Protocolo de llamada de la sección 7.
`param_in`	Lee el argumento desde `$fp + F + 8·i`.
`return`	Valor en `$v0`/`$v1`, promoción al tipo de retorno y salto al epílogo.
`new_list`, `length`	`jal __new_list`; lectura de la longitud en la cabecera con verificación de `null`.
`load_index`, `store_index`	`jal __index`, que verifica límites y devuelve la dirección del elemento.
`new_object`	Reserva la cabecera y los campos con `__alloc_zero`, guarda el nombre de la clase y llama al inicializador de la clase.
`load_field`, `store_field`	Verifica `null` y accede a `base + 8 + offset`.
`try_begin`, `try_end`, `catch`	Apila o retira un registro de manejador (sección 10).

Tabla 8: Traducción de cada familia de cuádruplos.

Las operaciones `begin_program`, `end_program`, `capture` y `static_link` no emiten código: su información ya quedó reflejada en la organización de rutinas y en los enlaces de acceso. Una ope-

ración sin traducción produce el diagnóstico `MIPS_UNSUPPORTED_TAC` en lugar de un ensamblador incompleto.

8.1. Ejemplo: una suma

Para el cuádruplo `%t0 = %a#2 add %b#3` dentro de una función, con los registros vacíos, el generador emite:

```
# TAC 4: %t0 = %a#2 + %b#3
addiu $t8, $fp, 64        # getReg(%a#2): par $s1/$s2
lw $s1, 0($t8)
lw $s2, 4($t8)
move $a0, $s1
move $a1, $s2
addiu $t8, $fp, 72        # getReg(%b#3): par $s3/$s4
lw $s3, 0($t8)
lw $s4, 4($t8)
move $a2, $s3
move $a3, $s4
jal __add                 # aritmética según etiquetas
nop
move $s1, $v0             # getReg(%t0): el destino queda en registro
move $s2, $v1             # y se marca como modificado
```

Listing 3: Traducción de una suma (offsets ilustrativos).

El resultado no se escribe en memoria de inmediato: queda en el par `$s1`/`$s2` marcado como modificado y solo se vuelca a su slot si el par se necesita para otro valor o antes de una llamada o un salto.

8.2. Entorno de ejecución

Al final de `.text` el generador agrega un conjunto de rutinas auxiliares escritas en ensamblador. Concentran las operaciones que dependen de la etiqueta de tipo y mantienen compacto el código de cada cuádruplo:

Rutina	Responsabilidad
`__alloc_zero`	Reserva memoria con `sbrk`, alineada a 8 y en cero.
`__new_list`, `__index`	Creación de listas y cálculo de dirección con verificación de límites.
`__add` … `__mod`, `__to_float`	Aritmética entera y flotante, promoción y empaquetado de dobles.
`__compare`, `__eq` … `__ge`	Comparación numérica mixta y de cadenas.
`__concat`	Concatenación de cadenas UTF-8.
`__print`, `__print_list`	Impresión de cada tipo, incluidas listas anidadas, objetos y `null`.
`__raise`, `__uncaught`	Lanzamiento de errores y terminación con código 2 si nadie los captura.

Tabla 9: Rutinas auxiliares del entorno de ejecución.

El segmento `.data` contiene el manejador activo, un slot por variable global (`__global_<id>`), los literales de cadena (`__str_n`) y flotantes (`__flt_n`) sin duplicados, y los mensajes del entorno.

9. Asignación de registros

9.1. Registros disponibles

Registros	Uso
`$s1`/`$s2`, `$s3`/`$s4`, `$s5`/`$s6`	Tres pares de la caché de valores: carga útil y etiqueta.
`$s0`	Receptor `this` de la rutina.
`$a0`–`$a3`	Operandos de las rutinas auxiliares; `$a0` transporta además el enlace de acceso en las llamadas.
`$v0`/`$v1`	Resultado de rutinas y funciones.
`$t0`–`$t9`	Registros de trabajo para direcciones y rutinas auxiliares.
`$fp`, `$sp`, `$ra`	Frame, pila y retorno.

Tabla 10: Asignación fija de los registros MIPS.

9.2. Política de `getReg`

El generador mantiene un **descriptor por par de registros**: qué valor contiene, si está modificado respecto de su slot (*dirty*) y cuándo se usó por última vez. `getReg(v)` aplica, en orden:

1. Si algún par ya contiene `v`, lo reutiliza sin emitir carga.
2. Si hay un par vacío, lo elige.
3. Si no, elige el **menos recientemente usado** (LRU).
4. Si el par elegido está modificado, emite un *spill*: guarda carga útil y etiqueta en el slot estable del valor anterior.
5. Si `v` se va a leer, lo carga desde su slot o desde el literal.

```
int Generator::getReg(const std::string &v, bool reading, int avoid) {
  for (int n = 0; n < 3; ++n)
    if (cache[n].value == v && !v.empty()) { cache[n].age = ++age; ... return
        n; }
  int n = -1;
  for (int j = 0; j < 3; ++j) if (j != avoid && cache[j].value.empty()) { n =
      j; break; }
  if (n < 0) for (int j = 0; j < 3; ++j)
    if (j != avoid && (n < 0 || cache[j].age < cache[n].age)) n = j;
  spill(n); cache[n] = {v, false, ++age};
  if (reading) load(v, p(n), k(n));
  return n;
}
```

Listing 4: Núcleo de `getReg` en `mips_generator.cpp`.

El parámetro `avoid` impide que el segundo operando de una operación binaria desaloje al primero. Antes de cada llamada, etiqueta y salto se ejecuta `flush`: se vuelcan los pares modificados y se invalidan los descriptores, de modo que la memoria refleja los valores que deben estar disponibles en cualquier camino de control que converja en ese punto.

9.3. Relación con los temporales del TAC

La asignación física es independiente del reciclaje de nombres `%tN` que realiza el TAC. Un mismo nombre temporal puede representar valores distintos en instrucciones sucesivas; como la etiqueta de tipo viaja con el valor, el reciclaje no confunde al backend. Cada nombre tiene un único slot por frame, así que la presión de registros se resuelve con *spills* y no requiere reservar memoria adicional.

9.4. Métricas

`MipsResult` expone `instruction_count` (instrucciones del ensamblador generado, incluido el entorno), `register_allocations` (selecciones de pares hechas por `getReg`) y `spills` (guardados efectivos). Estas métricas describen el código estático; no son el número dinámico de instrucciones que ejecuta MARS. El IDE las muestra en la pestaña *Código MIPS*.

Esta es una asignación **local y conservadora con respaldo en pila**: no intenta una asignación global por coloreo de grafos, que el enunciado no exige, y privilegia la corrección en llamadas y en los puntos donde convergen caminos de control.

10. Errores de ejecución y `try/catch`

La fase anterior conservaba `try/catch` como operaciones abstractas porque no existía un ABI de excepciones. Esta fase lo define. La palabra `__handler` en `.data` apunta al manejador activo. Un `try_begin` reserva 64 bytes en la pila y guarda allí:

- el manejador anterior (para encadenar manejadores anidados);
- la etiqueta del `catch`;
- `$fp` y `$sp` en el momento de entrar al `try`;
- los registros `$s0`–`$s7`.

Cuando una rutina auxiliar detecta un error (división por cero, índice fuera de rango, referencia nula, resultado no finito) salta a `__raise`, que restaura `$fp`, `$sp` y los registros del manejador activo, deja el mensaje como valor del `catch` y salta a su etiqueta, aunque el error haya ocurrido varias llamadas más abajo. Si no hay manejador, `__uncaught` imprime *Error de ejecución:* seguido del mensaje y termina con el servicio 17 y código 2.

Para que los manejadores no queden colgados, cada frame guarda el manejador de entrada (offset 44) y lo restaura en el epílogo, y los saltos que salen de un `try` —`return`, `break`, `continue`— retiran antes los registros de manejador que abandonan.

11. Integración con MARS y código máquina

11.1. Preparación del simulador

`scripts/setup_mars.py` descarga el JAR oficial de MARS 4.5 a `tools/Mars.jar` y verifica su SHA-256. Las variables `MARS_JAR` y `JAVA` permiten usar una instalación existente. Los scripts de construcción preparan MARS antes de configurar CMake.

11.2. Ejecución

`mips_runtime.py` ejecuta MARS sin interfaz gráfica:

```
java -jar tools/Mars.jar nc sm ae65 se70 1000000 programa.asm
```

Listing 5: Comando de referencia de MARS.

`nc` elimina el aviso inicial, `sm` comienza en la etiqueta global `main`, `ae65` y `se70` asignan códigos de salida distintos a los errores de ensamblado y de simulación, y el entero limita los pasos ejecutados (entre 32 y 10 000 000), lo que protege al IDE frente a ciclos infinitos. El reporte distingue `assembly` de `execution` y conserva `stdout`, `stderr`, `returncode`, `timed_out` y `step_limit`.

11.3. Código máquina

El adaptador puede ensamblar sin ejecutar y empaquetar el resultado en un ZIP:

Archivo	Contenido
`program.asm`	Ensamblador del programa y de su entorno de ejecución.
`program.text.bin`	Volcado binario del segmento de instrucciones ensambladas.
`program.data.bin`	Volcado binario del segmento de datos iniciales.
`manifest.json`	Arquitectura, simulador, direcciones y tamaños de segmentos, comando de ejecución.
`LEEME.txt`	Instrucciones para abrir y ejecutar el programa en MARS.

Tabla 11: Contenido del paquete de código máquina.

Los binarios son volcados sin cabecera ELF; para reproducir la ejecución se usa `program.asm`, que permite a MARS reconstruir direcciones, datos y código de forma coherente.

12. Interfaz gráfica (IDE)

El IDE es la misma aplicación web servida por Flask de las fases anteriores. El botón principal ahora se llama *Compilar programa* y la barra de fases muestra *ANTLR → semántica → TAC → ensamblador MIPS32 → MARS*. La fase agrega:

- **Pestaña *Código MIPS***: el ensamblador generado, sus métricas (instrucciones, selecciones de registros y *spills*), la tabla de frames físicos y botones para copiar y descargar el `.asm` o el ZIP de código máquina.
- **Botón *Ejecutar en MIPS* y pestaña *Consola MIPS***: ensambla y ejecuta el programa en MARS y muestra la salida estándar, los errores de ensamblado o de ejecución y el código de retorno, separados de los diagnósticos del compilador.
- **Estado del simulador**: indica si Java y `Mars.jar` están disponibles antes de intentar ejecutar.

Ruta	Función
`POST /api/analyze`	Compila y devuelve el JSON de todas las fases.
`GET /api/simulator`	Disponibilidad de Java y MARS.
`POST /api/run`	Compila y ejecuta en MARS; devuelve salida y errores.
`POST /api/build`	Ensambla y devuelve el ZIP de código máquina.
`POST /api/tests`	Ejecuta la batería C++ desde la interfaz.

Tabla 12: Rutas del adaptador Flask.

Si el programa tiene errores léxicos, sintácticos o semánticos, la tarjeta *Código MIPS* indica *Omitido* y la ejecución no se intenta: el IDE nunca envía a MARS código de un programa rechazado.

13. Batería de pruebas

La fase se verifica en tres niveles complementarios, con **193 casos automatizados**:

Batería	Casos	Qué verifica
`compiscript_tests.cpp`	115	Semántica, TAC y omisión de MIPS ante errores, con el código de diagnóstico exacto.
`mips_execution_tests.py`	66	Genera ensamblador desde `.cps`, lo ejecuta en MARS y compara la salida con un resultado esperado: 54 válidos y 12 errores esperados.
`ide_integration_tests.py`	12	Rutas Flask con el compilador y MARS reales.
Total	**193**	

Tabla 13: Baterías de pruebas del proyecto.

13.1. Pruebas de ejecución en MARS

La batería `mips_execution_tests.py` **no interpreta Compiscript ni TAC**: los resultados observados proceden del ensamblador ejecutado por MARS. Las salidas esperadas se calcularon a partir de los programas fuente, no del TAC, y cada caso válido exige además un volcado binario no vacío de `.text`, para comprobar que el ensamblado produjo código máquina. Los casos negativos distinguen un diagnóstico esperado de un fallo inesperado del ensamblador.

Categoría	Casos	Ejemplos
Proceso completo	1	Programa vacío.
Expresiones	5	Precedencia, promoción flotante, comparaciones mixtas, resto flotante.
Control de flujo	6	Cortocircuito con efectos laterales, ciclos, `break`/`continue` anidados, `switch`.
Llamadas y retornos	8	Factorial, Fibonacci, recursión mutua, seis parámetros, retorno flotante.
Asignación de registros	2	Temporales vivos alrededor de llamadas, presión con doce argumentos.
Orden de evaluación	3	Argumentos de izquierda a derecha, operando conservado ante efecto lateral.
Ámbitos y closures	7	Sombreado, dos niveles léxicos, funciones hermanas, `this` en función anidada.
Cadenas	4	Concatenación, orden lexicográfico, escapes, UTF-8.
Listas	5	`foreach`, listas anidadas, paso por referencia.
Clases y objetos	7	Constructores, herencia, inicializadores, receptor preservado.
Algoritmos del compilador	1	`algoritmo_compilador.cps`.
Errores de compilación	7	Léxico, sintáctico y semántico impiden generar ensamblador; aridad.
Errores de generación	1	Literal entero fuera de rango MIPS32.
Errores de ejecución	4	División por cero, índices fuera de rango, referencia nula.
Excepciones	5	`try/catch` entre llamadas, manejadores anidados, `return` y `continue` dentro de `try`.
Total	**66**	

Tabla 14: Casos de ejecución en MARS por categoría.

13.2. Prueba de fuego: algoritmos del propio compilador

`algoritmo_compilador.cps` adapta a Compiscript dos algoritmos del repositorio y recorre con ellos el pipeline completo hasta MARS:

- `DFABuilder::bestRule` (`backend/src/dfa_builder.cpp`): elige la regla aceptante de menor prioridad y usa cortocircuito para evitar una lectura fuera de rango.
- `TacGenerator::newTemp` y `releaseTemp` (`tac_generator.cpp`): administran temporales vivos con una pila de libres y contadores de creación, reutilización y máximo vivo.

```
function mejorRegla(posiciones: integer[], reglas: integer[], total: integer)
    : integer {
  let mejor: integer = -1;
  foreach (posicion in posiciones) {
    if (posicion < total && reglas[posicion] >= 0) {
      if (mejor == -1 || reglas[posicion] < mejor) {
        mejor = reglas[posicion];
      }
    }
  }
  return mejor;
}
```

Listing 6: Fragmento de `algoritmo_compilador.cps`.

La adaptación usa listas y una clase; la pila tiene capacidad fija de ocho posiciones porque el lenguaje no define `push`, `pop` ni listas redimensionables. No es autocompilación de los archivos C++: la sintaxis Compiscript sigue siendo el contrato de entrada. La salida esperada, `2, -1, 7, 0, 1, 0, 2, 2, 3, 2, 3` (un valor por línea), se compara con la de MARS.

13.3. Ejecución de las baterías

```
ctest --test-dir build/compiscript --output-on-failure
python3 backend/tests/mips_execution_tests.py
python3 backend/tests/mips_execution_tests.py --filter "Llamadas" --json
python3 backend/tests/ide_integration_tests.py
```

Listing 7: Ejecución de las tres baterías.

CTest registra la batería MIPS cuando al configurar CMake encuentra Python, Java y `tools/Mars.jar`. Las pruebas del IDE requieren instalar antes `requirements.txt`.

14. Compilación y ejecución

14.1. Requisitos

- CMake 3.20 o superior.
- Compilador compatible con C++17.
- Java, para la herramienta generadora de ANTLR y para el simulador MARS.
- Python 3.10 o superior con Flask, para el IDE y el adaptador de MARS.

14.2. Construcción

Un solo script descarga el JAR de ANTLR si hace falta, regenera el lexer, el parser y el Visitor, prepara y verifica MARS, compila el *runtime* C++, construye `compiscript_cli` y `compiscript_tests` y ejecuta las baterías C++ y de ejecución MIPS:

```
python3 scripts/setup_mars.py
./scripts/build_compiscript.sh
```

Listing 8: macOS o Linux.

```
py scripts/setup_mars.py
.\scripts\build_compiscript.ps1
```

Listing 9: Windows PowerShell.

14.3. Ejecución del IDE

```
python app.py
```

Listing 10: Servidor del IDE.

El IDE queda disponible en `http://127.0.0.1:5050`. Se abre un ejemplo, se pulsa *Compilar programa*, se revisa la pestaña *Código MIPS* y se pulsa *Ejecutar en MIPS* para ver la salida en la *Consola MIPS*.

14.4. Uso directo del CLI

```
./compiscript_cli --file programa.cps --emit-mips -o programa.asm
python3 scripts/run_mips.py programa.cps -o programa.asm
python3 scripts/run_mips.py programa.asm
python3 scripts/run_mips.py programa.cps --assemble-only --package programa.
    zip
```

Listing 11: Generación y ejecución desde la línea de comandos.

Sin `-emit-mips`, el CLI conserva la salida JSON con los bloques `tokens`, `tree`, `semantic`, `tac` y `mips` (`text`, `diagnostics`, `frames`, `instruction_count`, `register_allocations`, `spills`). `run_mips.py` acepta `-json` para el reporte estructurado, `-max-steps N` para el límite de instrucciones y `-delayed-branches` para activar los saltos retardados del simulador.

15. Decisiones, supuestos y limitaciones

- **Solo se genera MIPS para programas válidos.** Léxico, sintaxis, semántica y TAC deben terminar sin errores; si no, el bloque `mips` se marca como omitido.
- **Valores con etiqueta de tipo.** Cada slot mide 8 bytes (carga útil y etiqueta). Duplica el espacio de un entero, pero permite reciclar temporales, imprimir cualquier valor y capturar errores de tipo desconocido.
- **Dobles por referencia.** Mantienen uniforme el tamaño del slot a costa de una reserva dinámica por resultado flotante calculado.
- **Despacho estático.** Los métodos se resuelven con el símbolo determinado por el análisis semántico y se llaman directamente; la herencia conserva campos y búsqueda de miembros, sin tabla virtual.
- **Closures limitados a frames activos.** Las funciones anidadas acceden a los frames léxicos mientras están en la pila; la gramática no define funciones de primera clase que escapen de su activación.
- **Saltos sin retardo.** Se emite un `nop` tras cada salto, de modo que el código es correcto con o sin la opción `-delayed-branches` de MARS.
- **Orden de evaluación.** Los argumentos se evalúan de izquierda a derecha y cada resultado se conserva antes de evaluar el siguiente.

Limitaciones conocidas.

- La asignación de registros es local: los descriptores se invalidan en cada etiqueta, salto y llamada, lo que produce cargas y *spills* que una asignación global evitaría.
- No hay optimización: no se pliegan constantes y cada operación aritmética llama a una rutina auxiliar que inspecciona las etiquetas, aun cuando los tipos son conocidos en compilación.
- No hay recolector de basura: la memoria dinámica se conserva hasta que termina el programa.
- La pila de manejadores y la memoria dinámica dependen de los servicios de MARS; ejecutar en hardware real requeriría sustituir esos servicios.

16. Correspondencia con la rúbrica

Componente	Pts	Evidencia en el proyecto
Llamadas y retornos	20	Registros de activación físicos con enlaces de control y acceso, prólogo y epílogo, paso de parámetros por pila, retorno en `$v0`/`$v1`, preservación de registros (sección 7); 8 casos de llamadas y 7 de ámbitos y closures ejecutados en MARS (sección 13).
Asignación de registros	20	`getReg` con descriptores, LRU, *spills* y `flush` en fronteras de control; métricas observables en el JSON y el IDE (sección 9); casos de presión de registros y temporales vivos entre llamadas.
Código MIPS	60	`MipsGenerator` de 1 012 líneas que traduce todas las familias de TAC (sección 8), entorno de ejecución, `try/catch` (sección 10), ejecución real en MARS y paquete de código máquina (sección 11); 66 programas comparados automáticamente.
Total	**100**	

Tabla 15: Evidencia por componente evaluado.

Respecto a los entregables: el repositorio conserva commits individuales por integrante (sección 4); las baterías cubren casos exitosos, errores de compilación, de generación y de ejecución (sección 13); la arquitectura y las instrucciones de ejecución están documentadas (secciones 3 y 14, además de `docs/generacion-mips.md`, `docs/arquitectura-compiscript.md` y `README.md`); y el IDE compila, descarga y ejecuta los programas (sección 12).

17. Conclusiones

1. Trabajar sobre cuádruplos estructurados, y no sobre su texto, permitió que el backend reutilizara toda la información de las fases anteriores: bastó con agregar los tipos de los operandos al TAC para seleccionar instrucciones y coerciones sin volver a analizar el programa.

2. Representar cada valor con una etiqueta de tipo resolvió de una sola vez el reciclaje de temporales, la impresión polimórfica y la variable del `catch`, a cambio de duplicar el tamaño de los slots.

3. Separar padre léxico y llamante en el enlace de acceso fue la decisión clave para que funciones anidadas, hermanas y recursivas encontraran su entorno correcto.

4. Un `getReg` local con respaldo en pila es suficiente para producir código correcto en llamadas y en puntos de convergencia, y sus métricas hacen visibles los *spills* que una asignación global eliminaría.

5. Ejecutar el código en MARS y comparar la salida con resultados calculados desde el programa fuente convierte la verificación estructural de la fase anterior en una verificación de comportamiento: el compilador produce programas que realmente funcionan.

18. Referencias

- Aho, A., Lam, M., Sethi, R. y Ullman, J. *Compilers: Principles, Techniques, and Tools.* 2.ª edición, 2006. Secciones 7.2.2–7.2.3 (registros de activación y secuencias de llamada), 7.3.5–7.3.6 (enlaces de acceso) y 8.6.1–8.6.3 (descriptores de registros, generación de código y `getReg`).
- Vollmar, K. y Sanderson, P. *MARS: MIPS Assembler and Runtime Simulator*, versión 4.5. Línea de comandos: `https://dpetersanderson.github.io/Help/MarsHelpCommand.html`. Servicios del sistema: `https://dpetersanderson.github.io/Help/SyscallHelp.html`.
- Parr, T. *The Definitive ANTLR 4 Reference*. Documentación del *target* de C++: `https://github.com/antlr/antlr4`.
- Repositorio del proyecto, rama `Compiladores2-Proyecto3-MIPS`: `https://github.com/G1LB3T0/Proyecto1_Compis_Generador_Analizador_Lexico/tree/Compiladores2-Proyecto3-MIPS`.
- Documentación interna del repositorio, en la carpeta `docs/`: `generacion-mips.md`, `codigo-intermedio-tac.md`, `arquitectura-compiscript.md` y `decisiones-compiscript.md`.